# Practical 4

**Aim: Develop a game-playing agent using minimax and alpha-beta pruning for Tic-Tac-Toe or a similar turn-based game.**

```
import random

board = [" "] * 9

def show(board):
    print()     for row_start
in [0, 3, 6]:
        a, b, c = board[row_start:row_start + 3]
print("  " + a + " | " + b + " | " + c)         if
row_start < 6:
            print("  --+---+--")
print()


LINES = [
    [0, 1, 2], [3, 4, 5], [6, 7, 8],
    [0, 3, 6], [1, 4, 7], [2, 5, 8],
    [0, 4, 8], [2, 4, 6]
]


def winner(board):      for a, b, c in LINES:          if board[a]
!= " " and board[a] == board[b] == board[c]:
            return board[a]
    return None


def empty_boxes(board):
    return [i for i in range(9) if board[i] == " "]
```

```
show(board) print("Board and
helpers ready.")


# Minimax algorithm
def minimax(board, turn):
win = winner(board)

   if win == "X":         return
1     if win == "O":
return -1     if not
empty_boxes(board):
return 0

   if turn == "X":
      best = -2

      for box in empty_boxes(board):
board[box] = "X"            score =
minimax(board, "O")
board[box] = " "            best =
max(best, score)

      return best
    else:
best = 2

      for box in empty_boxes(board):
board[box] = "O"            score =
minimax(board, "X")
board[box] = " "            best =
min(best, score)

      return best


def best_move(board):     best_score
= -2     best_box =
empty_boxes(board)[0]

   for box in empty_boxes(board):
board[box] = "X"         score =
```

```
minimax(board, "O")
board[box] = " "

      if score > best_score:
best_score = score          best_box
= box

   return best_box


print("The AI can now look ahead and choose its best move.")


def simple_opponent(board):
return empty_boxes(board)[0]


# Play against a simple opponent
board = [" "] * 9 turn = "X"

print("The game begins! X is our smart AI, O takes the first free box.")
show(board)

while winner(board) is None and empty_boxes(board):
if turn == "X":
      move = best_move(board)
board[move] = "X"       print("AI
(X) plays box", move)       turn =
"O"    else:
      move = simple_opponent(board)
board[move] = "O"       print("Player
(O) plays box", move)       turn = "X"

   show(board)

who = winner(board)
```

```
if who == "X":
   print("Result: The AI (X) WINS!") elif
who == "O":
   print("Result: O wins.") else:
   print("Result: It is a draw.")


# Test Minimax against a random opponent
wins = 0 draws = 0 losses = 0

for game_number in range(30):
   board = [" "] * 9
turn = "X"

   while winner(board) is None and empty_boxes(board):
if turn == "X":            board[best_move(board)] = "X"
turn = "O"        else:
          board[random.choice(empty_boxes(board))] = "O"
turn = "X"

   result = winner(board)

   if result == "X":
wins += 1     elif
result == "O":
losses += 1     else:
       draws += 1

print("Out of 30 games against a random player:")
print("AI wins :", wins) print("Draws   :", draws)
print("AI loses:", losses)
print("Wins and draws may change each run.")


# Alpha-Beta pruning algorithm def
alphabeta(board, turn, alpha, beta):
   win = winner(board)

   if win == "X":
return 1
   if win == "O":
```

```
        return -1      if not
empty_boxes(board):
return 0

    if turn == "X":
        best = -2

        for box in empty_boxes(board):
            board[box] = "X"            score =
alphabeta(board, "O", alpha, beta)
board[box] = " "

            best = max(best, score)
alpha = max(alpha, best)

            if alpha >= beta:
                break

        return best

else:
        best = 2

        for box in empty_boxes(board):
            board[box] = "O"            score =
alphabeta(board, "X", alpha, beta)
board[box] = " "

            best = min(best, score)
beta = min(beta, best)

            if alpha >= beta:
                break

        return best
```

```
print("The AI now knows the faster Alpha-Beta way.")


def best_move_ab(board):
    best_score = -2
    best_box = empty_boxes(board)[0]

    for box in empty_boxes(board):
        board[box] = "X"         score = alphabeta(board, "O",
float("-inf"), float("inf"))         board[box] = " "

        if score > best_score:
best_score = score             best_box
= box

    return best_box


# Play using Alpha-Beta pruning
board = [" "] * 9 turn = "X"

print("Same game, this time the AI uses Alpha-Beta pruning.") show(board)

while winner(board) is None and empty_boxes(board):
if turn == "X":
        move = best_move_ab(board)
board[move] = "X"         print("AI
(X) plays box", move)         turn =
"O"     else:
        move = simple_opponent(board)
board[move] = "O"         print("Player
(O) plays box", move)         turn = "X"

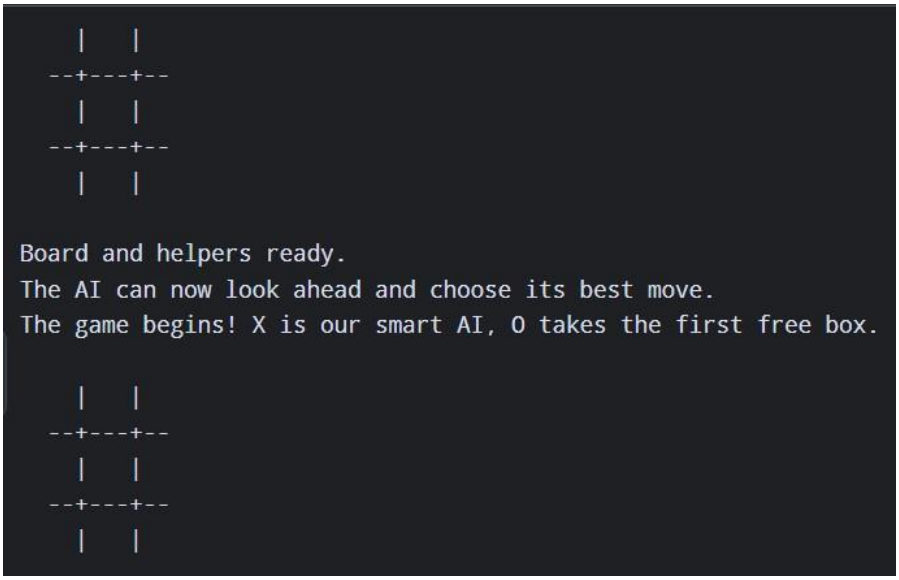
    show(board)

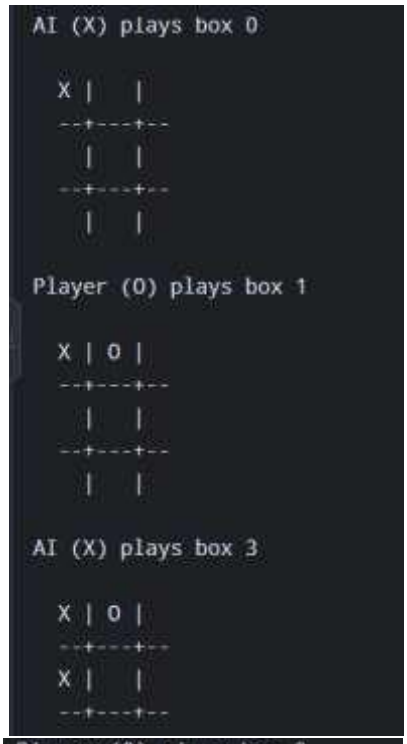
who = winner(board)

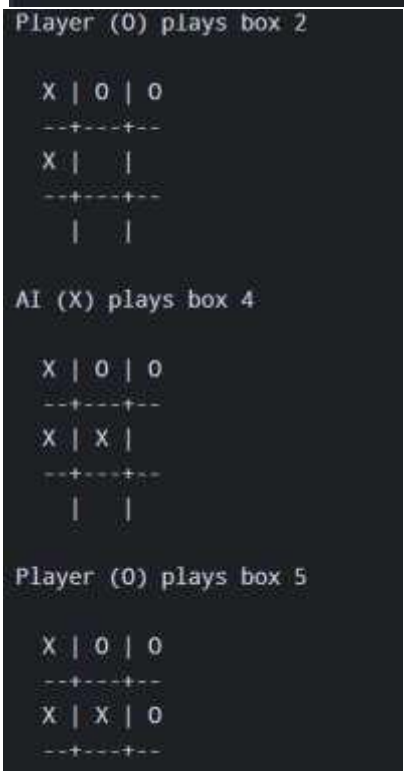
if who == "X":
    print("Result: The AI (X) WINS!") elif
who == "O":
    print("Result: O wins.")
```

else: print("Result: It is a draw.")

**OUTPUT**




AI (X) plays box 0
X | |
Player (O) plays box 1
X | O |
AI (X) plays box 3
X | O |
X | |


Player (O) plays box 2
X | O | O
X | |
AI (X) plays box 4
X | O | O
X | X |
Player (O) plays box 5
X | O | O
X | X | O


AI (X) plays box 6
X | O | O
X | X | O
X |
Result: The AI (X) WINS!
Out of 30 games against a random player:
AI wins : 30
Draws : 0
AI loses: 0
Wins and draws may change each run.
The AI now knows the faster Alpha-Beta way.
Same game, this time the AI uses Alpha-Beta pruning.

```
AI (X) plays box 0

  X |   |
  --+---+--
    |   |
  --+---+--
    |   |

Player (O) plays box 1

  X | O |
  --+---+--
    |   |
  --+---+--
    |   |

AI (X) plays box 3

  X | O |
  --+---+--
  X |   |
  --+---+--
```

```
Player (O) plays box 2

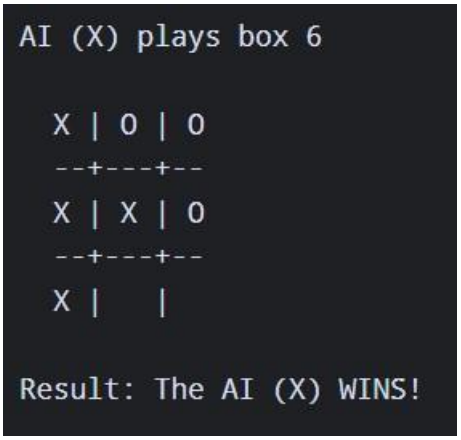
  X | O | O
  --+---+--
  X |   |
  --+---+--
    |   |

AI (X) plays box 4

  X | O | O
  --+---+--
  X | X |
  --+---+--
    |   |

Player (O) plays box 5

  X | O | O
  --+---+--
  X | X | O
  --+---+--
```




# Practical 5

**Aim: Build a small rule-based expert system or logic inference engine for a diagnosis or advisory problem.**

```
def laptop(turns_on, internet_works, is_slow):
if turns_on == "no":        return "Restart it"
```

```
elif internet_works == "no":        return
"Airplane Mode"     elif is_slow == "yes":
        return "Let it start properly"
else:
        return "No problem"


turns_on = input("Does the laptop turn on? ").lower()
internet_works = input("Does the internet work? ").lower() is_slow
= input("Is the laptop slow? ").lower()

print("Suggestion:", laptop(turns_on, internet_works, is_slow))
```

**OUTPUT**

```
Does the laptop turn on? yes
Does the internet work? yes
Is the laptop slow? yes
Suggestion: Let it start properly
```

# Practical 6

**Aim: Solve a constraint satisfaction problem such as map coloring, Sudoku, or N-Queens using backtracking and consistency checking.**

```
map_data = {
   'A': ['B', 'C'],
   'B': ['A', 'C', 'D'],
```

```
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}

colors = ['Red', 'Green', 'Blue']


def is_consistent(region, color, assignment):        for neighbor in
map_data[region]:                    if neighbor in assignment and
assignment[neighbor] == color:              return False      return True


def solve_map(assignment):
    # Check if all regions are colored
if len(assignment) == len(map_data):
        return assignment

    # Select the next uncolored region
for region in map_data:          if
region not in assignment:
            break

    # Try each available color      for color in
colors:          if is_consistent(region, color,
assignment):
            assignment[region] = color

            result = solve_map(assignment)
if result is not None:
                return result

            # Backtrack
            del assignment[region]

    return None
```

```
solution = solve_map({})

print("Solution:") if solution is not
None:     for region, color in
solution.items():
      print(region, "=", color)
else:     print("No solution
exists")
```

## OUTPUT:

```
Solution:
A = Red
B = Green
C = Blue
D = Red
```